

CVORTEX – TEAM PROJECT

“In our Resume Analyzer, my part starts after the resume text has already been extracted by my teammate. I take that resume text from the backend and send it to the LLaMA 3.3 70B model through the Groq API.

The important part of my contribution was the prompt. I created a detailed prompt that tells the AI to analyze the resume like an ATS and resume reviewer. I specify exactly what information I want, such as the overall score, strengths, weaknesses, ATS compatibility, missing skills, section-wise feedback, and suggested improvements.

I also instructed the model to return the result in a fixed JSON format. This is important because the frontend needs structured data to display things like scores, lists, and feedback.

After getting the AI response, my backend parses the JSON and sends the structured result back to the React frontend. I also added error handling in case the AI response is not valid JSON or the Groq API fails.

So basically, the flow is: resume text → my prompt + resume text → LLaMA through Groq → structured JSON response → backend → React frontend.”

If they ask: “What exactly is your prompt doing?”

You can say:

“The prompt gives the model instructions about what to analyze and how to return the result. Instead of simply asking ‘analyze this resume’, I define the exact sections I want, such as score, strengths, weaknesses, ATS issues, matched skills, missing skills, and improvement suggestions. I also specify the JSON structure so the frontend can directly use the response.”

If they ask: “Does the AI actually understand the resume?”

Say:

“It's not understanding the resume like a human does. LLaMA is a language model that processes the text and predicts the most appropriate response based on patterns it learned during training. My prompt guides it to focus on specific resume-related criteria and produce the structured analysis we need.”

If they ask: “Did you train the AI model?”

Be very clear:

“No, I didn't train the model. We used the pre-trained LLaMA 3.3 70B model through Groq's API. My contribution was integrating the model into our backend, designing the prompt, controlling the output structure, and connecting the result to the frontend.”

★ Most important sentence to remember

“My main contribution was not building the LLaMA model itself; it was integrating the existing model into our application and using prompt engineering to get a structured, useful resume analysis that the frontend could directly display.”

MZPLACENEXT-COLLEGE PROJECT

MZPLACE Next is an AI-powered interview practice platform that I developed specifically for my college. Actually, this project started from my personal project called Pinky AI, which I initially built as a small interview practice application. I showed that project to my college, and they asked me to develop a similar platform specifically for our students.

Through this project, I got the opportunity to think more like a real developer. Initially, my application worked fine for one or two users, but when we planned to use it with many students at the same time, I started thinking about things like API limits, token usage, request limits, concurrency, and how to handle failures.

The platform allows students to upload their resume and practice interviews. The resume is analyzed and Gemini generates interview questions based on the student's resume. We have four rounds — Resume Screening, Technical, System Design, and Behavioral. Each round has three attempts, and each attempt contains 15 questions.

The student answers the questions by speaking. The audio is sent to Deepgram for speech-to-text conversion, and the transcript is stored in the database. Once the round is completed, the questions and answers are sent to Gemini again, which evaluates the student's performance and generates scores, strengths, weaknesses, and feedback.

One of the biggest technical challenges was handling the AI API limits. Initially, I used Groq, but its free-tier limits were not suitable for our expected usage. So I moved to Gemini. I learned about RPM, TPM, input and output tokens, and how API usage changes when multiple students use the system simultaneously. I also implemented multiple API keys with a request-handling approach to distribute the AI requests and reduce the chance of hitting a single key's limit.

We tested the platform with multiple students, and this project taught me a lot about building an AI application that is not just functional, but also needs to be reliable when real users access it.”

1. Frontend — React

The student interacts with the application through the React frontend.

They can:

- Upload their resume
- Start an interview
- View questions
- Record their answers
- Move through attempts
- View scores and feedback

The frontend communicates with the backend using API requests.

Simple interview explanation:

“I used React for the frontend, where students interact with the interview platform. The frontend communicates with the Express backend through APIs.”

2. Backend — Node.js + Express

The backend is responsible for handling the main application logic.

It manages:

- Resume processing
- Question generation
- Interview sessions
- Audio processing
- AI evaluation
- Database operations

So the frontend doesn't directly communicate with Gemini, Deepgram, or the database.

React

↓

Express Backend

↓

AI / Database / Speech Services

This also keeps API keys and sensitive operations on the server.

3. Resume → Gemini

Once the resume is available, the backend sends the relevant resume information to Gemini.

The prompt tells Gemini what type of interview questions to generate.

For example:

Resume

+

Interview Round

+

Prompt



Gemini



Questions

The questions are generated based on the student's background rather than being completely generic.

4. Four Interview Rounds

The application has four rounds:

Round 1 — Resume Screening

Questions are mainly based on:

- Resume
- Projects
- Skills
- Experience

Round 2 — Technical

Questions focus on the student's technical knowledge.

For example:

- Programming
- DSA
- Technologies
- Database
- Development concepts

Round 3 — System Design

Questions test how the student thinks about designing systems.

Round 4 — Behavioral

Questions focus on communication, situations, teamwork, and decision-making.

5. Three Attempts Per Round

Each round has **3 attempts**.

Each attempt has **15 questions**.

So:

1 Round



3 Attempts



15 Questions each



45 Questions

This allows students to practice the same round multiple times without simply repeating the exact same set of questions.

6. Gemini AI Integration

This is one of the important parts of your project.

You initially used **Groq**, but when you started thinking about actual college usage, you realized that API limits were an important issue.

You then moved to **Gemini**.

You learned about:

RPM

Requests Per Minute

How many API requests you can send within a minute.

TPM

Tokens Per Minute

How many input + output tokens you can process within a minute.

Tokens

AI doesn't process text exactly like humans count words.

Text is divided into smaller units called **tokens**.

For example:

Resume text



Input tokens



Gemini



Output tokens

You started monitoring this because your application makes multiple AI requests.

7. Multiple API Keys

This is one of the things you should mention because it shows your **scaling thinking**.

Your application uses multiple Gemini API keys.

Instead of sending every request through one key:

100 requests



API Key 1

you distribute requests:

└─ API Key 1

Requests ───┐

└─ API Key 2

The idea is to distribute usage across separate quota pools.

Interview explanation:

“When I started testing with multiple students, I realized that using a single API key could hit the request limits quickly. So I implemented multiple API keys and distributed requests between them to make better use of the available quota.”

Don't say **“I increased Google's limit.”**

You didn't increase the limit—you distributed requests across separately configured keys/quota pools.

8. Student Answers — Audio Recording

After Gemini generates the questions, the student answers them through the browser.

Instead of typing:

Question



Student speaks



Audio recording

The audio is sent to your backend.

9. Deepgram — Speech to Text

The backend sends the recorded audio to **Deepgram**.

Deepgram converts:

Student's voice



Deepgram



Text transcript

For example:

Student says:

“I used MongoDB because it is flexible and works well with...”

Deepgram returns the corresponding text.

That transcript is then stored in the database.

10. Database — SQL Server

You use the database to maintain the student's interview information.

Conceptually:

Student



Interview Session



Round



Question

↓

Answer / Transcript

↓

Feedback

This allows the system to remember the student's progress and results.

11. AI Evaluation

After the student completes a round, the backend collects:

Questions

+

Student Answers

and sends them to Gemini.

The prompt tells Gemini to behave like an interviewer/evaluator.

Gemini analyzes things such as:

- Correctness
- Quality of answer
- Strengths
- Weaknesses
- Communication
- Improvements
- Score

Then it returns the evaluation.

12. Structured AI Response

One important thing you learned from your AI projects is that you can't simply ask:

“Evaluate this student.”

You need to tell the AI exactly what output you want.

For example:

{

score: 8,

```
    strengths: [...],  
    weaknesses: [...],  
    feedback: [...],  
    improvedAnswer: [...]  
}
```

This makes it easier for the React frontend to display the result.

13. Why This Project Was Important for You

This is actually the **strongest part of your story**.

Your Pinky AI project was your starting point.

Pinky AI



Personal Project



Showed to College



College asked for a platform



MZPLACE Next



Real Students



Real Usage Problems



Scaling + API Limits + Tokens + Reliability

You can say:

“The biggest difference between Pinky AI and MZPLACE Next was that Pinky AI was mainly a personal project. With MZPLACE Next, I had to think about what would happen when many students actually use the system. That changed the way I approached development.”

That's a **very good interview story**.

★ If Recruiter Asks: "What Did You Learn From This Project?"

Say:

"This project taught me to think beyond just making a feature work. When it became a real college application, I had to think about API limits, token usage, RPM, TPM, concurrent users, error handling, and reliability. I also learned how to monitor AI usage and optimize API calls. So this project helped me move from thinking like a student building a project to thinking more like a developer building something for real users."

That's probably your **best answer** for this project.

Your entire project in one sentence:

"MZPLACE Next is an AI-powered interview practice platform where students upload their resume, Gemini generates personalized interview questions, students answer through voice, Deepgram converts their answers to text, Gemini evaluates their performance, and the frontend displays the scores and feedback—with the backend handling the AI, database, API limits, and overall workflow."